

Blockchain et autorité de certification

Janvier 2020

Table des matières

1	Présentation	1
1.1	Principe d'une blockchain	1
1.1.1	Le minage	2
1.1.2	Contenu des blocs	2
1.1.3	Arbre de Merkle [Merkle tree]	3
1.1.4	Sécurité de la blockchain	4
1.1.5	Les transactions	4
1.1.6	Vérification	4
1.1.7	Qui veut la peau de la blockchain ?	4
1.1.8	Problème de la certification et de la vérification de l'identité	4

Le programme C va simuler d'une autorité de certification "décentralisée" à l'aide d'une blockchain. En fait si une blockchain de type Bitcoin est vraiment décentralisée c'est qu'elle est gérée par peer to peer. Ici nous allons implémenter une blockchain centralisée mais on va faire comme si.

1 Présentation

1.1 Principe d'une blockchain

La blockchain ou chaîne de blocs est une structure de données informatique qui permet de sécuriser des données, supposées être publiques, contre toute tentative de modification ou altération. Pour cela on utilise un algorithme de hash code qui permet d'associer à une donnée une valeur numérique unique. Si H est la fonction qui implémente un de ces algorithmes, et x une donnée quelconque, on a :

- $H(x)$ donne toujours la même valeur pour le même x
- $H(x) \neq H(x')$ même si la différence entre x et x' est minime.
- connaissant $H(x)$ est très difficile de retrouver x .

L'algorithme de hash que nous utiliserons est le SHA256 (nombres de 256 bits). Nous allons manipuler son résultat sous forme d'une chaîne de caractères hexadécimaux. La longueur de la chaîne est fixe (32 octets, donc caractères).

La blockchain est constituée de blocs chaînés entre eux sous forme de liste. Jusque là ça reste une SD très classique en informatique. Pour éviter qu'un bloc disparaisse sans laisser de trace, on va ajouter (signer) chaque bloc avec le hash code de son contenu. Et pour qu'il laisse une trace en cas de disparition on va inclure dans le contenu du bloc le hash code de son prédécesseur. Et donc, le hash code du bloc va intégrer le hash code du bloc précédent dans la chaîne.

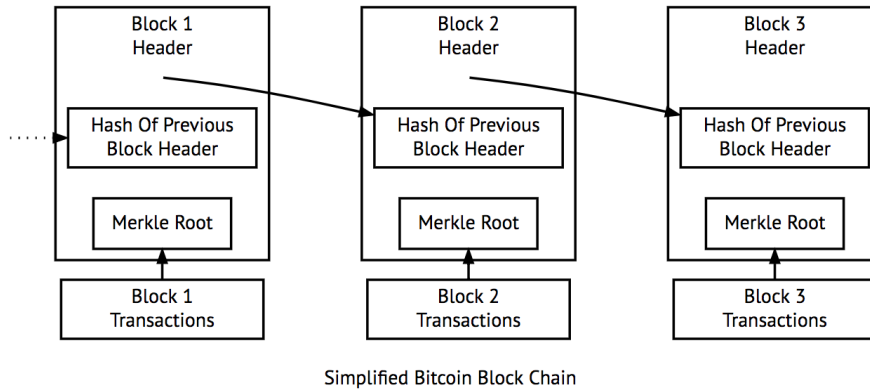


FIGURE 1 – La blockchain

Que se passe-t-il si on veut supprimer un bloc quelque part au milieu de la chaîne ? De toute évidence on perturbe la continuité des hash codes. Il faut donc recalculer le hash de tous les blocs suivant celui qui disparaît et refaire le chaînage. La sécurité de la chaîne tient ici à la quantité de travail à faire pour altérer la chaîne.

Même sur un grand nombre de block, on peut imaginer pouvoir le faire avec un gros PC. Ça reste jouable me direz-vous. On va donc pimenter le tout avec la "difficulté". Pour faire simple, disons que la difficulté est un critère que doit remplir le hash code et qui assure qu'il n'est pas trop facile à calculer même avec une machine performante. Dans la pratique (Bitcoin) la difficulté est adaptée à l'évolution de la puissance des machines. Elle est choisie pour qu'il faille 10mn pour la calculer. A itérer autant de fois qu'il y a de blocs à recalculer pour altérer la chaîne sans que cela ne se voit. Comme un nouveau bloc arrive toutes les 10mn, il faudrait recalculer l'ensemble de la chaîne en 10mn pour que cela passe inaperçu (ou presque car il y a aussi d'autres mécanismes dans un univers peer2peer où il y a autant de copies de la chaîne que de noeuds dans le réseau).

Nous allons simplifier ce mécanisme en choisissant une difficulté représentée par un entier d , et en demandant que le hash du bloc commence par d zéros dans sa représentation en ascii hexadécimal. Et si cela n'est pas le cas ?

1.1.1 Le minage

On introduit dans le contenu de chaque bloc un entier initialisé à zéro et appelé "nonce". Si le hash du bloc (incluant la nonce) ne satisfait pas le critère, on incrémente la nonce et on recommence le calcul du hash. On répète ce processus jusqu'à ce que le hash satisfasse le critère (la difficulté). Ce calcul est appelé «minage» [mining].

1.1.2 Contenu des blocs

Que met-on dans les blocs ? Ce que l'on veut en fait. Le Bitcoin gère des transactions d'argent (crypto-monnaie [crypto-currency]). Nous allons imiter ce principe bien qu'il soit annexe dans ce sujet. Donc il y aura des transactions de token du genre «Astérix envoie 10 token à Obélix». Il y a un nombre variable de transactions dans un bloc qui est normalement (Bitcoin) dépendant de la taille maximale du bloc et de celle des transactions. Nous allons fixer arbitrairement le nombre maximal de transactions (constante du programme). De plus, le bloc contient la date de création (timestamp).

Le premier bloc de la chaîne est particulier. Il ne peut pas intégrer le hash de son prédécesseur car il n'a pas de prédécesseur. On l'appelle bloc «généซิส». Le hash de son prédécesseur est zéro, sa nonce reste à

zéro, il a une unique transaction réduite à la chaîne « Genesis »¹. Par contre il a bien un hash. Il n'est pas nécessaire de le miner.

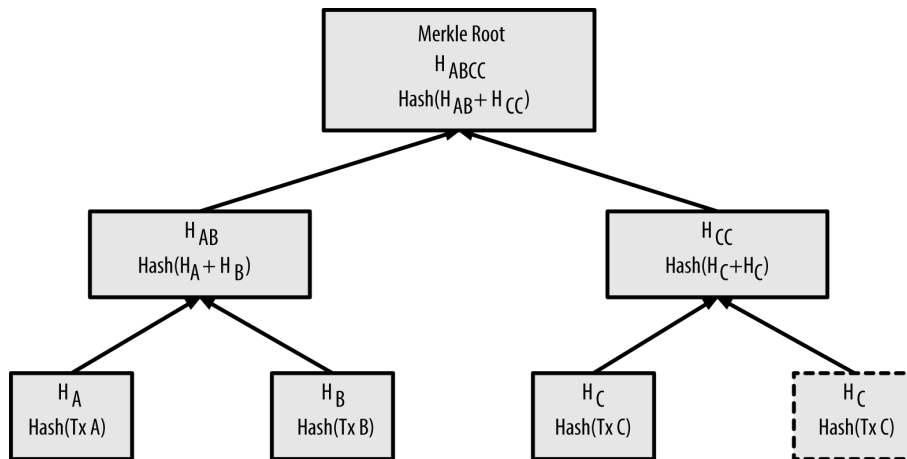
Que se passe-t-il si on veut altérer une transaction d'un bloc sans toucher aux hash de ce bloc ?

1.1.3 Arbre de Merkle [Merkle tree]

Si on altère une transaction d'un bloc sans toucher aux hash de ce bloc, le hash du bloc va quand même être modifier puisque qu'il prend en compte le contenu de tout le bloc, et la liste des transactions fait partie du contenu (pour notre sujet car le Bitcoin est légèrement différent).

Pour corser le tout on va calculer l'arbre de Merkle des hash des transactions :

1. supposons que nous ayons quatre transactions : $t1, t2, t3, t4$,
2. on calcule les hash de chaque transaction. On obtient $h1, h2, h3, h4 = H(t1), H(t2), H(t3), H(t4)$
3. puis on calcule le hash de $h1$ concaténé avec $h2$ et le hash de $h3$ concaténé avec $h4$. Soit $h1.2$ et $h3.4$ les résultats,
4. puis on calcule $H(h1.2 \text{ concaténé } h3.4)$, soit $h1.2.3.4$ le résultat.



Le résultat final $h1.2.3.4$ est le racine de l'arbre [Merkle tree root]. C'est un hash code. Ce hash sera inclus dans le contenu du bloc.

Si le nombre de transactions (ou de hash intermédiaires) est impair, on duplique simplement la dernière transaction (ou hash).

Connaissant les transactions qui sont écrites en clair on peut recalculer la racine de l'arbre et vérifier que c'est bien le même hash root que celui qui apparaît dans le bloc, et que donc, aucune transaction n'a été altérée

1.1.4 Sécurité de la blockchain

Quelques attaques (dans la cas de notre blockchain simplifiée) :

- Supprimer ou modifier une transaction : Cela modifie le merkle root et donc le sha du block et provoque une inconsistance de la chaîne.
- Rejouer une transaction déjà présente. Normalement le timestamp et le numéro de tx doit empêcher ça car il sont dans le hash de la signature. Il faudrait modifier ces données. De plus cela modifie le merkle tree. Le test de hash root le détecte.

1. Pour reprendre le nom donné à ce block par Satoshi Nakamoto (聡中本) l'inventeur du Bitcoin.

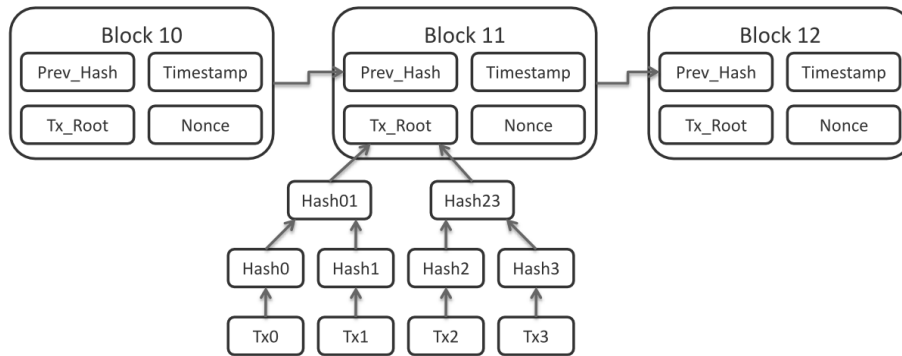


FIGURE 2 – Merkle tree

- Agir sur les blocks. Supprimer un block pour faire disparaître une transaction. Cela supprime aussi les autres transactions. Et le chaînage de hash est perturbé. Il faut tout recalculer.

1.1.5 Les transactions

Les transactions seront des documents à archiver par la blockchain :

- déclaration de clef de cryptage ou de signature associée à une identité
- déclaration de révocation de clef

Pour créer une transaction, c'est à dire enregistrer un document, une personne doit envoyer une requête. Nous ferons l'impasse sur la vérification de l'identité de l'émetteur car cette phase est délicate. Chaque transaction intègre, de plus, une date (timestamp).

1.1.6 Vérification

L'intégrité de la blockchain peut être vérifiée à tout moment.

1. On vérifie que la chaîne commence bien par le bloc génésis et que le chaînage des hash est valide, et que le hash du bloc est bien celui annoncé (vérification 1).
2. On vérifie pour chaque bloc que le hash Merkle root correspond bien aux transactions de ce bloc (vérification 2).

1.1.7 Qui veut la peau de la blockchain ?

Bien sûr notre blockchain n'est pas inaltérable, ni celle du Bitcoin d'ailleurs. Puisque que la nôtre est centralisée on peut faire ce que l'on veut avec ce qui n'est pas le cas de celle du Bitcoin qui est répartie et soumise au principe du consensus (mais c'est une autre histoire).

1.1.8 Problème de la certification et de la vérification de l'identité

Astérix est un Youtubeur connu mais qui a négligé les autres réseaux sociaux et ne connaît rien à la cryptographie. Il n'a donc pas de compte FB ni insta ou twitter. Acidecloridrix (nom de code HCL), l'espion de César, ouvre un compte mail [astérix@gmail.com], envoie une tx à la blockchain avec des adresses crypto et signature qu'il a créés, ouvre des comptes FB et insta avec cette adresse mail, communique en se faisant passer pour Astérix et incite les gens à utiliser ses adresses crypto.

Assurancetourix, qui vient de créer l'oeuvre de sa vie, un opéra épique et magnifique, au dire de son auteur, décide de l'envoyer à Astérix pour avoir son avis. Il fait confiance à Astérix, qu'il connaît par ailleurs, mais décide d'utiliser la cryptographie pour sécuriser la transmission de son message. Il cherche les adresses crypto d'Astérix sur la BC, trouve celle envoyée par HCL, et ne fait pas attention à l'adresse mail qui n'est pas celle qu'il connaît. Il envoie donc son opéra, crypté avec la clef d'HCL à l'adresse créée par HCL, en

croyant correspondre avec Astérix. HCL peut donc lire le message et envoie l'opéra à César, ceo de Rome and co limited, une multinationale sans scrupule déjà condamnée pour avoir spoilé de nombreux auteurs.